

Secret-Key Sampling Watermarks for Carbon

Genomic LM: Research Plan

Executive Summary. *Carbon* is a family of autoregressive language models for DNA, using a **hybrid tokenizer**: standard BPE for text and fixed non-overlapping 6-mers for DNA, triggered by `<dna>` tags [23†L230-L238] [57†L206-L214]. Carbon-500M is a 0.5B “draft” model (used in speculative decoding), Carbon-3B is the flagship 3B model, and Carbon-8B is an 8B model. All share the same 6-mer DNA tokenizer and `<dna>...</dna>` syntax [47†L200-L208] [57†L206-L214]. 6-mer tokens encode exactly 6 DNA bases (A,C,G,T) each. We will **not** retrain Carbon, but rather **modify generation/sampling** via a secret key so that the output DNA contains a hidden trace (watermark) attributable to the key. The detector uses **only the output DNA and the verification key** (plus known algorithm), and must work under edits (substitutions, indels, cropping, reverse-complement).

This document details every design and implementation step. We begin by carefully *auditing Carbon*: verifying model/checkpoint details, hybrid tokenizer behavior (including `<dna>`, `<oov>`, partial padding, etc.), and how generation actually samples from the vocabulary. From that we define **baselines** for watermarking, including (B0) the raw full-vocab distribution, (B1) DNA-only renormalized distribution, (B2) actual Carbon decoding policy (temp/top-k/top-p with DNA filtering), and (B3) the Carbon *FNS* (Factorized Nucleotide Supervision) mode that generates base-by-base. We compare standard token-level (4096-way) vs FNS (6×4-way) generation in detail.

We then design **keyed samplers**: at minimum (SAMP-A) binary partition-and-couple, (SAMP-B) keyed inverse-transform sampling, and (SAMP-C) keyed exponential-minimum sampling [50†L28-L33]. For each we specify the PRF-based partitioning or randomness, how the latent bit is tied to vocabulary subsets, and how to preserve the baseline distribution. We also review related schemes (DiPmark, unbiased watermarking, etc.) and weigh their applicability.

We plan a fully standalone **detector** that, given DNA and a key, searches all 12 orientation/phase hypotheses (6 frames × {forward, reverse-complement}), realigns into non-overlapping 6-mers, recovers watermark bits, and applies statistical tests (with multiple-testing correction) to detect the watermark. We discuss normalization (e.g. case, whitespace, IUPAC codes), tokenization of fragments, and strategies for handling indels (global sync, blocks, anchors, dynamic alignment, etc.).

We model the *channel*: secret codeword → keyed sampler → (intrinsic noise) → 6-mer stream → (DNA errors: subs/ins/dels/cropping/reverse) → observed streams → detector errors. We quantify *per-base substitution vs per-token watermark error rates*: e.g., if nucleotide substitution prob = p , then any given 6-mer is corrupted with prob $1-(1-p)^6$. We give numerical examples and a running example:

```
Hidden bits:      1 0 1 1 0 0 1 0 ...
Carbon probs:    AACGTA(0.08), GCTTAC(0.07), ... (see table below)
Partition sets:  S0/S1 →  $q_t \approx 0.6$  → hid bit=0 (S1)
```

```
Generated tokens: TTGGCA | CGATGC | ...
Obs. bits:          1 0 1 0 0 0 1 0
```

(illustrated with mismatches and reasons).

We define **synchronization strategies**: from naive fixed-phase (SYNC-0) to robust methods: fixed-size blocks (SYNC-1), sliding windows (SYNC-2), using known DNA anchor motifs (SYNC-3), dynamic-programming alignment (SYNC-4), and ultimately error-correcting coding (SYNC-5).

Finally, we specify **experiments (E0–E13)**: from synthetic sampler tests (E0), Carbon integration and channel measurement (E1–E2), distribution-preservation checks (E3), clean detection (E4), substitution/indel/crop robustness (E5–E7), sync method comparison (E8), null/wrong-key calibration (E9), unkeyed distinguishability (E10), adversarial removal (E11), baseline sampler comparisons (E12), to integrating ECC/PRC (E13). For each we give data schemas (Parquet tables with fields like `sequence_id`, `position_bp`, `entropy_full`, etc.), metrics (entropy, bits/token, TPR/FPR, edit rates, AUC), and target scale ($\geq 10^6$ positions for channel, long DNA runs for detector). We include a **one-week feasibility sprint** outlining Day1–Day7 tasks and deliverables, plus a provisional 12-week roadmap (see Tables).

At the end we present a **decision framework**: possible outcomes are *CONTINUE* (6-mer channel viable), *CONTINUE BUT REDESIGN SYNCHRONIZATION* (if indels kill global phase), *USE GENERIC WATERMARK* (if existing method suffices), *USE FNS INSTEAD* (if base-level channel is better), or *STOP* (if failure modes prevail, e.g. no usable entropy or detection fails).

The plan is grounded in **primary sources**: the Carbon technical report and model repos, the HF `carbon` repo, each Carbon model card (500M/3B/8B), the hybrid tokenizer code, the FNS code path, the HF Transformers generation code, and the cited watermark literature [23†L230–L238] [50†L28–L33] [51†L29–L37] [54†L23–L31]. All versions (model, tokenizer, Transformers, PyTorch, CUDA) will be pinned, and commit SHAs documented in the Manifest tables. The final report will serve as a blueprint for implementation: it contains pseudo-code (for example, Python-style function specs for loading Carbon, building distributions, sampling, and detection), UML/mermaid diagrams (architecture, pipelines, timelines), tables (vocab manifest, who-knows-what, experiments, thresholds), and references.

Main Repositories/Commits Inspected: (Carbon repo, model cards & tokenizer)

- *huggingface/carbon* (tech-report.pdf) – GitHub [Commit 95c3c68] [21†L182–L189].
- *HuggingFaceBio/Carbon-3B* (main, commit **7a95b29** at writing) [20†L239–L247] [23†L226–L234].
- *HuggingFaceBio/Carbon-3B (fns branch)* (same commit plus FNS code) [9†L455–L462] [47†L278–L287].
- *HuggingFaceBio/Carbon-500M* (main, revision matching 3B usage) [47†L191–L200] [47†L278–L287].
- *HuggingFaceBio/Carbon-8B* (main, commit **3e1df85** per HF UI) [57†L206–L214] [57†L239–L247].
- Carbon tokenizer code (above references [23–25]).
- Transformers v4.** pinned (the version supporting `trust_remote_code` for Carbon) – examine `GenerationMixin` logic.
- Sampling baseline code: e.g. `GenerationMixin._do_sample_logits` (transformers code) for injection points.

- Carbon evaluation suite (for distribution tests).

Tokenization Audit (Sections 6-9). Carbon's **hybrid tokenizer** (a subclass of `PreTrainedTokenizer`) wraps Qwen3 BPE and DNA 6-mers [23†L226-L234]. In DNA mode (`<dna>...</dna>`), text between tags is chunked into *non-overlapping* 6-base blocks. All canonical DNA bases (`A,C,G,T`) are uppercased. The code builds:

- **DNA special tokens:** `"<dna>"`, `"</dna>"`, and `"<oov>"`, assigned IDs starting at `dna_start_id = base_vocab_size` [23†L285-L294].
- **Canonical 6-mer tokens:** all $4^6=4096$ sequences of `A,T,C,G`, enumerated in lexicographic order of the base list `['A','T','C','G']` [23†L288-L290]. They receive consecutive IDs. (This list is used as `self.kmers = [''.join(kmer) for kmer in itertools.product(bases, repeat=6)]` [23†L288-L290].)
- **Padding tokens:** to align the vocab size to a multiple of 128 (practical for efficient GPU embedding lookup), a few `<unused_i>` tokens are appended [23†L300-L309].
- **Mappings:** `dna_token_to_id` and `dna_id_to_token` map between token strings and IDs (includes specials, kmers, padding) [23†L308-L317] [23†L326-L329]. The combined vocab merges base+BPE vocab and DNA vocab with no ID collisions (even if a DNA 6-mer coincides with a BPE string) [23†L293-L302] [23†L333-L342]. The resulting vocab size (max ID+1) is divisible by 128. For example, with Qwen3's base vocab of 151936 tokens [41†L1-L4] plus 4099 DNA tokens, we align to 156,032 tokens (3 padded tokens) in practice.

We will output a **vocabulary manifest** (e.g. `artifacts/carbon_vocab_manifest.parquet`) with rows for each token:

```
token_id | token_string | token_class | is_canonical_6mer | is_special |
is_metadata | is_text_bpe
```

where e.g. token `"AAAAAA"` is `token_class=DNA`, `is_canonical_6mer=1`, and BPE vs special fields accordingly. All 4096 canonical 6-mers should appear exactly once. We will exhaustively verify:

- Every string in `{A,C,G,T}^6` maps to a unique token ID.
- Decoding round-trips: for any 6-mer, `decode([id(token)]) == token`.
- No overlaps: tokenization yields adjacent 6-mer tokens that exactly reconstruct the DNA.
- The DNA mapping is identical in 500M/3B/8B and in the FNS branch (FNS reuses the same tokenizer).

For example (pseudocode):

```
tok = AutoTokenizer.from_pretrained("HuggingFaceBio/Carbon-3B",
trust_remote_code=True)
dna_ids = {tok.token_to_id[kmer] for kmer in all_6mers}
assert len(dna_ids)==4096
for kmer in all_6mers: assert tok.decode([tok.token_to_id[kmer]]) == kmer
```

We will include representative sample tests in `tests/tokenizer_tests.py`.

<dna> Mode and OOV: The tokenizer only enters 6-mer mode inside `<dna>...</dna>` tags [23†L230-L238] . Without tags, a DNA-like string (e.g. "ACGTAA") would be tokenized using Qwen3 BPE (likely as separate subwords). The code split logic (`_split_by_dna_tags`) ensures any text outside `<dna>` is handled by BPE [24†L364-L372] [24†L396-L404] . The `auto_dna_tags` option (default `False`) can wrap untagged input, but we will usually manually insert `<dna>` . During encoding:

- If `<dna>` is present, the segment parser `_parse_dna_region` strips `<dna>` / `</dna>` and calls `_process_dna_sequence` [24†L422-L430] [24†L434-L442] .
- In `_process_dna_sequence` , the DNA sequence is uppercased (`dna_seq = dna_seq.upper()`) [24†L430-L438] . Hence lowercase a/c/g/t are normalized (treated as their uppercase).
- The sequence is then chunked every 6 bases: for each block of 6, if all characters $\in \{A, T, C, G\}$, it is added as a canonical token; otherwise the token `"<ooov>"` is added [24†L437-L445] . For example, `'N'` , `'R'` , `'Y'` (IUPAC ambiguity) or lowercase `'a'` (which becomes `'A'`) are handled. Actually lowercase is converted to uppercase first, so `'a'` $\rightarrow$ `'A'` is valid [24†L430-L438] , but `'N'` $\rightarrow$ `'N'` which is not in $\{A, T, C, G\}$ triggers `<ooov>` [24†L438-L444] .
- If the sequence length is not a multiple of 6, the **final partial block** is right-padded with `'A'` 's to length 6 [24†L452-L460] . E.g. sequence "ACGT" becomes padded "ACGTAA" (padding length 2, valid length 4) and this padded k-mer is used if it contains only A/T/C/G; otherwise `<ooov>` . The code comments explain that padding is on the *end* (not the start) so that the first 4 bases are "valid" and supervised, the last 2 are "padding" [24†L452-L460] .
- The special tag `"</dna>"` is also emitted after the last k-mer token.

Thus in DNA mode, all tokens correspond to 6bp (or `<ooov>`). The returned **token mask** array (when `return_token_mask=True`) encodes lengths: e.g., a canonical k-mer gets mask=6, a partial k-mer gets mask=actual bases (1-5), and `<ooov>` tokens and special tags get mask=0 [24†L518-L526] . This mask could help the FNS loss, but for our sampler we mostly need the token IDs.

We must test corner cases:

- **Empty DNA:** `<dna></dna>` should probably produce an empty sequence or just no k-mers. The code would generate a `<dna>` then see empty `dna_content` , then immediately output `"</dna>"` . We should verify if this is allowed. Probably treat as no tokens (we can disallow empty DNA in prompts).
- **Non-canonical chars** (`N, R, Y` , whitespace, newline): Each k-mer containing any invalid char becomes `"<ooov>"` [24†L438-L445] . For example, "ACGNTA" $\rightarrow$ tokens ["ACGNT(A)" - `<ooov>` , "Axxxxx" -pad-> if valid then, else `<ooov>`]. Lowercase letters become uppercase and are valid.
- **Partial lengths:** lengths mod6=1..5 are padded as above. For example "ACGTTA GCACT" (10 letters) yields tokens for "ACGTTA" and "GCACTA" (last padded with one `A` to "GCACTA"). The mask would show 6 and 5. We must decide: will the watermark sampler treat these partial tokens differently? Ideally, we either avoid prompting odd lengths or incorporate the mask logic (maybe simpler to restrict to multiples of 6 in generation). The report will note how to handle/truncate/pad such cases.
- **<ooov> token generation:** If Carbon ever produces an `<ooov>` token (very low probability except if prompt had invalid), treat it as a special event. Watermark should ensure `<ooov>` is either suppressed (by disallowing in sampling) or handled.

- **Tokenizer state:** The model's `transformers` integration ensures that after a `<dna>` token, the next tokens are interpreted in DNA mode until `</dna>` appears [24†L514-L522] [25†L575-L584]. We must check if the model can ever generate `</dna>` on its own. The code does append `</dna>` automatically if needed on encode, but in generation, once `<dna>` starts, the model can only output within DNA until generating `</dna>`. Likely it can output `</dna>` (it's a token in the vocab). If it does, the tokenizer leaves DNA mode and any subsequent bases would be tokenized by BPE again [25†L575-L584]. We should test if `<dna>` mode can accidentally "leak" (e.g. generation without `</dna>` and then bump into `max_length`, etc.). For watermarking, we will assume generation remains inside `<dna>` throughout (i.e. user ends prompt with `<dna>` and we keep generating DNA tokens, finally `</dna>` or end-of-sequence). Our sampler should either enforce DNA-only sampling or be prepared to handle any special token with probability.

Full Vocabulary Mass Measurement (Sec. 10). At each generation step t , Carbon outputs logits over the *entire hybrid vocabulary* (base+BPE + 6-mers + specials). We will empirically measure how much probability mass is on DNA 6-mers versus BPE versus metadata/special tokens. Define: $m_{\text{DNA},t} = \sum_{z \in \mathcal{V}_{\text{6mer}}} P_t(z)$, $m_{\text{BPE},t} = \sum_{z \in \mathcal{V}_{\text{BPE}}} P_t(z)$, $m_{\text{special},t} = \sum_{z \in \{\text{metadata, specials, oov, pad}\}} P_t(z)$. If Carbon is in `<dna>` mode, ideally $m_{\text{DNA},t} \approx 1$, but it may assign nonzero to e.g. `<oov>`. We will log these to see if any significant mass is wasted (and if we need to re-normalize). Similarly in BPE/text mode, $m_{\text{BPE},t} \approx 1$ [23†L230-L238]. Understanding this informs whether to use B0 (full distribution) or restrict to DNA tokens (B1).

Baselines (Sec. 11,16–17). We define mathematically:

- **B0 (Full Carbon):** The model's original distribution $P_t^{\text{full}}(z)$ over all tokens (logits after temperature, top-k/p etc). This is how vanilla Carbon generates (with text and DNA tokens mixed). This baseline might occasionally pick BPE tokens in DNA mode if not clamped.
- **B1 (DNA-only renormalized):** Restrict to the 4096 canonical DNA tokens, renormalizing: $P_t^{\text{DNA}}(z) = \frac{P_t(z)}{\sum_{u \in \mathcal{V}_{\text{6mer}}} P_t(u)}$ for $z \in \mathcal{V}_{\text{6mer}}$. This yields a pure DNA channel. But note this *changes* the distribution if some mass was on others. We will compare B1 to B0 to gauge effect.
- **B2 (Deployed Carbon):** This is the actual deployed policy, e.g. with top-k, top-p, temperature, and with non-DNA tokens either allowed or filtered. For example, we may decide to set `do_sample=True, top_k=40, top_p=0.95, temp=1.0, filter_special_tokens=False`, and possibly explicitly zero out `<oov>` or non-DNA tokens. The resulting final categorical $P_t(z)$ (after these processors) is the distribution we truly preserve. B2 is what we consider *the baseline distribution* for watermarking.
- **B3 (Carbon FNS):** In the FNS branch, generation is conceptually 6 independent 4-way draws (one per nucleotide) rather than a single 4096-way draw [47†L278-L287] [57†L239-L247]. In practice, Carbon's FNS code draws 6 letters from distributions and returns their product distribution over kmers. We will audit and treat FNS as an alternate baseline: either separate or we map it back to a 6-mer distribution $P_t^{\text{FNS}}(z) = P(b_1) \cdots P(b_6)$ for $z = b_1 \dots b_6$. FNS is a useful control: it allows a direct base-level watermark (6 bits per token max) and may be more resilient to edits. We'll compare FNS vs standard thoroughly.

Transformer Generation Pipeline (Sec. 14–15). Carbon models derive from `LlamaForCausalLM`, so generation follows the `transformers.GenerationMixin` sequence: compute logits, then apply logit processors (e.g. forced tokens, repetition penalties), then warpers (temp, top-k/p), then sample/argmax. We want to inject keyed sampling *after* final $P_t(z)$ is defined but before drawing Z_t . Possible integration points:

- **LogitsProcessor:** These receive raw logits and can zero out tokens (e.g. to ban `<ooV>`) [50†L536-L542]. Could implement a custom `LogitsProcessor` to modify logits based on key. But to preserve distribution exactly on average, a Warper or sampler is more precise.
- **LogitsWarper:** In HF4.** new `LogitsWarper` objects can adjust logits multiplicatively or additively. Not sure if they exist for our context.
- **Custom loop:** Easiest for control: manually do `model(...)`, then softmax to get P_t , then apply our keyed sampling code to pick Z_t , then feed it back. This gives full visibility. We will likely implement our own loop (option III) for clarity.
- **Custom GenerationMixin:** Harder, skip unless needed.

We must ensure **baseline and watermark share all preprocessing**. I.e. up to computing $P_t(z)$, everything is identical. Only the sampling of Z_t diverges.

We will define $\mathcal{H}_t$ as the history (all previous tokens, context, key state, etc) and insist that for any z ,
 $P_t(z_t = z \mid \mathcal{H}_t) \approx P_t(z_t = z \mid \mathcal{H}_t)$ averaged over watermark randomness. If we can prove exact equality (as distortion-free samplers promise [50†L28-L33]), even better. We label *greedy decoding* (top-1) as the zero-entropy limit; note if we set `do_sample=False`, watermark has no channel (capacity 0) because output is fixed. All our schemes assume `do_sample=True`.

Sampling Distribution Statistics (Sec. 18–19). Before designing watermark, we will characterize Carbon's natural entropy. We will sweep parameters (`top_k` $\in \{2, 4, 8, 16, 32, 64, \infty\}$, `top_p` $\in \{\dots\}$, `temp` $\in \{0.5, 0.8, 1.0\}$ etc) and measure per-token entropy and *effective support size*
 $V_{\text{eff}} = \exp(H(P_t))$, $V_{\text{support}} = |\{z: P_t(z) > 0\}|$,
 at many steps of Carbon-3B (in DNA mode). We expect extremely skewed distributions: often *one or few 6-mers dominate*, especially with low temp/top-k. The watermark capacity (bits per token) = expected mutual information $I(C_t; Y_t)$ where Y_t is the watermark bit observed. We convert to bits/base by dividing by 6. This tells us how long a fragment we need for detection. Carbon's typical decoding might concentrate on few kmers, limiting watermark entropy. We will record *top-1 mass*, *top-2 mass*, etc. We will also repeat on Carbon-500M/8B to see scale effect (bigger model might be *more* peaked or less; unclear).

Keyed Samplers (Sec. 20–23). We implement three core schemes:

- **SAMP-A: Binary Partition & Coupling.** Choose a secret keyed function $g_K(z) \in \{0, 1\}$ that partitions the 4096 tokens into two groups. For example, $g_K(z) = \{\text{rm LSB}\}(\text{PRF}(K, z))$ (a pseudorandom bit per token). At step t , compute $q_t = \sum_{z: g_K(z)=1} P_t(z)$, the probability mass of group-1. Then we generate a secret uniform random bit C_t (independent per token, known to detector via PRF and context). We couple: choose Z_t by sampling from P_t *conditioned* on $g_K(Z_t) = C_t$. Concretely, if $C_t = 1$, we sample from the renormalized distribution on group-1; if 0, from group-0. This ensures $\mathbb{P}(g_K(Z_t) = C_t) = 1$. Because $P(Z_t = z \mid C_t) =$

$\begin{cases} \frac{P_t(z)}{q_t} & \text{if } g_K(z) = C_t, \\ 0 & \text{otherwise,} \end{cases}$ and since $P(C_t=1)=q_t$, this exactly preserves $P_t(z)$ marginally [50†L28-L33]. We must manage the case $q_t=0$ or 1 (rare if partition is balanced). We will derive the mismatch rate (how often $g_K(Z_t) \neq g_K(\text{baseline } Z_t)$) and info rate $\mathbb{I}(C_t; Y_t) = H(C_t) - H(C_t | Y_t)$.

- **SAMP-B: Keyed Inverse-Transform Sampling (ITS).** As in Kuditipudi et al. [50†L28-L33]: fix an ordering of tokens (e.g. lexicographic). Use a PRF to generate a pseudorandom threshold $u \in [0,1]$ for each step (derived from key and context). Compute the cumulative distribution $F_t(z) = \sum_{u \leq z} P_t(u)$ in that order. Output the smallest z s.t. $F_t(z) \geq u$. To watermark, we can manipulate u so that $u \in [0,0.5]$ or $[0.5,1]$ depending on a code bit. Implementation: combine key and context to seed a PRF that yields $r_t \sim U(0,1)$, then let $u_t = (r_t + b_t/2) \bmod 1$ where b_t is the hidden bit. Detector, knowing key, recomputes u_t and inverts the CDF from the observed Z_t to deduce b_t . We will specify PRF(K,t) usage, how to split into bit and uniform. The detector statistic is difference between empirical CDF and 1/2 line. ITS is *exactly* distribution-preserving [50†L28-L33]. Main cost: requires ordering tokens and potentially scanning CDF.
- **SAMP-C: Exponential-Minimum Sampling.** As Kuditipudi also describe, draw independent random $\xi_{t,z} \sim \text{Uniform}(0,1)$ via PRF for each token, and compute for each z : $R_z = -\ln(\xi_{t,z}) / P_t(z)$. Then pick $Z_t = \arg\min_z R_z$. Equivalently: $Z_t = \arg\max_z P_t(z) \cdot (-\ln \xi_{t,z})^{-1}$. If $\xi_{t,z}$ are i.i.d. $\text{Exp}(1)$ random, this yields exactly one draw from the categorical P_t [50†L28-L33]. To watermark, we can shift the exponential by a small factor keyed by bit (see Kuditipudi). For instance, if hidden bit $b_t=1$, multiply all $P_t(z)$ in one partition by $(1+\epsilon)$ (plus renorm) which biases the min. Alternatively, use separate PRFs for $\xi_{0,z}, \xi_{1,z}$ depending on bit. We will derive a precise formulation to let C_t bias $\min$ choice while preserving P_t . The detector uses order statistics of the two processes to infer C_t .

We will also consider:

- **P0 (static partition):** $g_K(z) = \text{LSB}(\text{PRF}(K,z)) \bmod 2$. Detector uses the same g_K .
- **P1 (context-keyed static):** $g_{K,h}(z) = \text{LSB}(\text{PRF}(K, h, z))$. If we can reconstruct context h (e.g. recent DNA tokens) in the detector, this adds extra key.
- **P2 (adaptive balanced partition):** The generator could re-partition tokens each time based on P_t (e.g. greedily split heavy vs light to make $q_t \approx 0.5$), but detector cannot know P_t . We treat this as an upper-bound (oracle).

We will measure for random K : the distribution of q_t vs 0.5. If random g_K already yields $|q_t - 0.5|$ small (say < 0.05 most of the time), then static P0 may be fine. Otherwise one could attempt P1 with context, but that complicates detection.

For each sampler we derive **information rates**: e.g. for SAMP-A with fair 50-50 split, $\mathbb{I}(C_t; Y_t) = H_b(q_t)$ per token (binary entropy with bias q_t). We will compute average $\bar{\mathbb{I}}(C_t; Y_t)$ and also per-base $I_{\text{token}} = I_{\text{token}}/6$. We'll present both.

We will not blindly implement all literature, but rank by: compatibility (standalone detector requirement), distribution-preservation guarantees, known implementations, and ease. For example:

- **Green/Red Partition (Sablayrolles et al.)** – essentially SAMP-A.
- **Gumbel-Max keyed sampling** (Trivedi et al.) – similar to exponential-min.
- **Permutation-based or DiPmark** – DiPmark modifies probabilities; it preserves distribution by design [51†L29-L37] but requires heavy reweighting (needs distribution knowledge). It’s accessible but complex; we will briefly discuss it but likely skip implementation.
- **Protein-watermark adaptation:** In private-key protein MPNN watermark (Vickers et al., 2023), a random binary string is injected by codon choices; if we treat each 6-mer like a “codon” for a 6-letter alphabet, analogies exist. We will ask: if we treat 6-mer tokens as “residues” from an alphabet of size 4096, is there a known scheme? Not directly, but the idea of mapping a random bit to token choices is similar to partition schemes. We include it as inspiration for P0/P1 approach.
- **Coding-theory methods (synchronization strings, edit-correcting)** – to be used later if basic sync fails. DNAMark/CentralMark (Khan et al. 2024) use synonymous codon changes for watermark in DNA, but they require rewriting coding regions, not applicable here (plus they break distribution preservation). We mention them as related *later* (likely in discussion).

Random Partition Balance (Sec. 24). We will simulate with actual Carbon-3B P_t and random g_K assignments: collect statistics of q_t . For instance, over 10^6 tokens, compute $|q_t - 0.5|$ at each step and its quantiles. If random is already ~ 0.5 on average, no need for advanced splitting.

Channel Information (Sec. 25). For a binary observed symbol Y_t , channel info $I(C_t; Y_t) = 1 - H(\text{BER})$, where $\text{BER} = \Pr(Y_t \neq C_t)$. We will compute this for each scheme (analytically if possible, else empirically via Monte Carlo). We then report information per *base*: $I_{\text{bp}} = I_{\text{token}}/6$, since 6bp consumed per symbol. This avoids inflated claims from using a 4096-alphabet as “space” – the real physical channel is nucleotides.

Randomness Table (Sec. 26). We differentiate randomness types:

Property	Model Uncertainty (P_t)	Watermark Code (PRF/key)	Sampling Randomness	
-----	-----	-----	-----	
				Secret? no (public model) yes (private key) maybe (if sampling random) Fresh per step? yes (changes each step) yes (PRF(kt)) yes (random draws) Detector knows? No (except public model) No (only key) No Attacker knows distribution? yes (model + context) no (key secret) no Reproducible? no (stochastic) yes (given key/nonce) no (if using <code>torch.multinomial</code>)

Knowledge Table (Sec. 27). We define our threat model: public knows Carbon weights, prompt, generation config, algorithm, but *not* the secret key or codeword. Detector knows key/algorithm, attacker does not. E.g.:

Information	Generator	Detector	Attacker
Carbon weights/models	✓	✓ (public)	✓
Model revision/checkpoint	✓	✓	✓
Prompt/meta tags	✓	✗ (not needed)	✗

Information	Generator	Detector	Attacker
Next-token logits	✓	✗ (not used)	✗
Decoding config (temp, top-k/p)	✓	✓ (we assume known)	✓
DNA output (suspect)	✓	✓	✗
Watermark algorithm/version	✓	✓	✓ (assume known)
Secret key	✓	✓	✗
Codeword	✗ (not known to attacker)	✓	✗
6-mer phase (start offset)	✓ (implicitly)	✗ (to detect)	✗
Generation offset/coordinate (for multi-block)	✓	✗ (detector finds)	✗

We will cite this in the report and ensure we follow the Kerckhoffs principle: only key is secret.

Running Example (Secs. 28–29). We will pick a toy sequence and illustrate step by step. For instance:

Prompt DNA (6-mer blocks): ACGTTA|GCACTG|TTAGGC|AACGTA...

Carbon next-step probs (illustrative):
AACGTA:0.08, GCTTAC:0.07, TTGGCA:0.06, CGATGC:0.05, AATTCC:0.04, ...

Suppose our keyed partition (PRF-based) assigns $AACGTA \in S_1$, $GCTTAC \in S_0$, $TTGGCA \in S_1$, $CGATGC \in S_1$, $AATTCC \in S_0$, ...

Then $q_t = P(S_1) \approx 0.08 + 0.06 + 0.05 + \dots = 0.6$.

Hidden bit C_t (PRF): say 0.

We sample from group-0 tokens (GCTTAC, AATTCC, ... renormalized). Suppose output $Z_t = \text{"GCTTAC"}$.

So observed bit $Y_t = g_K(Z_t) = 0$.

In the example table, we continue for 8 steps, listing: C_t (hidden), q_t , selected group, token sampled, etc.

Some steps will show mismatch ($C_t \neq Y_t$) if due to rounding or if Z_t happens to fall in group != bit (rare if implementation exact).

Finally we show:

Hidden codeword: 1 0 1 1 0 0 1 0
Observed: 1 0 1 0 0 0 1 0
Generated DNA: TTGGCA | CGATGC | ... (alignment to these tokens).

We will explain any mismatches (e.g. if two tokens had equal mass or rounding). This table clarifies sampling mechanics.

Detector Design (Secs. 30–33). The detector takes a **suspect DNA fragment** (string) and a verification key. It will:

1. **Normalize:** uppercase, strip whitespace/linebreaks, remove any `N` or non-ACGT (or map them to `<ooV>` in tokenization).
2. Generate two sequences to test: the original and its reverse complement.
3. For each orientation, for each of 6 possible phase offsets (0–5), parse the DNA into 6-mer tokens starting at that offset (drop incomplete ends if any). (Example in [29] covers that.)
4. For each hypothesis (orientation+phase), apply the Carbon tokenizer on the substring (just DNA, no `<dna>` needed) to get token IDs (or equivalently, map each 6-mer to the canonical token string, or `<ooV>`).
5. For each candidate token stream, derive the **observed bits**: $Y_t = g_K(Z_t)$ using the same partition or PRF used in generation (the key and context). The “context” here is critical: depending on the sampler, context may include the generator’s hidden state, but for partition/P0 we need not know context (just $PRF(K, token_id)$). For ITS/EXP, context matters. If context includes previous hidden bits or something, we must reconstruct or approximate it. Typically we assume PRF uses e.g. sequence position.
6. Align the observed bits to possible positions in the key’s codeword (which we assume is infinite or long enough). If synchronization strings/anchors are used, use them. Otherwise, compute a statistic over contiguous windows of bits. For example, for SAMP-A (simple partition), we test if $\sum_t (-1)^{Y_t} \approx 0$ (should concentrate around 0 if random, deviate if watermarked). For ITS/EXP, we compute the appropriate log-odds or permutation score (as defined in Kuditipudi). We calibrate the test against the null (random text) to get p-values.
7. Apply **multiple-testing correction**: since we test 12 hypotheses per window, we either take max-statistic or apply Bonferroni. Ultimately we output *YES/NO* based on a global threshold.

We will diagram the pipeline as a flowchart (mermaid) showing: DNA input → orientation/phase loop → 6-mer tokenization → hidden-to-observed bit mapping → sliding windows alignment → watermark score → final decision.

Phase Handling (Sec. 31–33). 6-mer tokens have an inherent phase (0=first base of first 6-mer). A random fragment could start in any of 6 phases. We will test all 6 and take the best (highest watermark score) accounting for multiple tests. Similarly try reverse-complement. For example, given `TTGGCA | CGATGC | GGAACC | ...`, if we remove 3 bases from start, parsing shifts. In [32] we illustrate this with a figure/table. The detector essentially solves a 12-hypothesis test. We assume attacker does not know which orientation/phase reveals watermark, so this is our cost.

Cropping (Sec. 32). If the fragment is *not* aligned to token boundaries, the 6-mer parse is ambiguous. By trying all 6 phases, we “slide” a 6-mer window over the DNA. For example, DNA `AAGTGGT | T...` cropping at offset 3 yields base sequence starting at 3rd base. We will show that one of the 6 phases will realign to the original tokens (if no errors). So cropping is a bounded search (6 possibilities). In practice we will search all and take the phase that maximizes the statistic (with correction).

Substitutions (Sec. 34–35). Each nucleotide change flips up to 6 bits. The probability a random 6-mer has ≥ 1 error under per-base error p is $1 - (1-p)^6$. For example, at $p=0.01$, this is $1 - (0.99)^6 \approx 0.058 \approx 5.8\%$. We will tabulate for $p=0.1\%, 0.5\%, 1\%, 2\%, 5\%$. But note: a token “error” doesn’t always flip the watermark bit. If $g_K(z)$ depends only on, say, the last base, some single-base errors won’t change $g_K(z)$. We will empirically measure the symbol-error rate $P(Y_t \text{ flips})$ under substitution. E.g., in our running example, if token `"AACGTA"` (group1) mutates to `"AACGTT"` (still in same group),

observed bit doesn't flip. But "AACGTA" → "ACCATA" might flip group. We'll simulate this to get $\$P(Y_t \neq Y'_t)\$$ given one base change. Example table [35] will show: before/after tokens, before/after group, thus $\$Y\$$ change.

Insertions (Sec. 36) and Deletions (Sec. 37). These cause misalignment. A single-base insertion shifts *all subsequent tokens*. E.g.:

Original 6-mers: TTGGCA | CGATGC | GGAACC | ...
 Insert A at start of second: TTGGCA | ACGATG | CGGAAC | C... (all downstream changed).

One insertion can ruin every following alignment. The detector must resynchronize. We will show a concrete DNA example step-by-step. Similarly for deletions. We will quantify: one insertion at rate $\$p\$$ yields expected downstream symbol error ~100% beyond that point.

Thus synchronization is key. We evaluate strategies:

- **SYNC-0 (global fixed phase):** assume start aligned; catastrophic on any indel (detect fails).
- **SYNC-1 (block codes):** if we watermark in independent small blocks (e.g. embed bit every 60bp independently), then indel only corrupts local block. We'd need redundancy and multiple blocks.
- **SYNC-2 (overlapping windows):** the detector could slide blocks of length L and test each (at all 12 alignments) and combine evidence. This is like multiple independent hypotheses – can mitigate indels if we take the best.
- **SYNC-3 (anchors):** find known DNA markers (e.g. common k-mers or metadata tags) to re-sync. For example, if the prompt includes a known species tag or pattern, use that as anchor. Possibly improbable in pure DNA gen.
- **SYNC-4 (dynamic alignment):** use DP (Needleman-Wunsch) to align observed bit sequence with expected watermark bit sequence, allowing indels, maximizing score. This is heavy but possible if sequences are shortish or using fast heuristics.
- **SYNC-5 (coding):** design the watermark bits with an index so that even if some bits drop, we can recover (synchronization strings, marker codes, ECC) [15†L1271-L1280]. For example, embed periodic markers. This is research-level complexity and done only if needed.

Our plan is to first implement simple (SYNC-2/3/4) and only design codes if we see need.

We depict the effective channel as:

```

flowchart TB
    subgraph Generator_Side [Generator Side]
        C[Codeword] --> S[Sampler[Keyed Sampler on P_t]]
        S --> Z[Z_t[6-mer tokens]]
    end
    subgraph DNA_Channel [DNA Channel]
        Z -- assemble --> D[DNA_seq[DNA string]]
        D --> A[Altered_DNA]
    end
    style Generator_Side fill:none,stroke:#ccc
    style DNA_Channel fill:none,stroke:#ccc
  
```

```

subgraph Detector Side
  Orientation/Phase --> TokenStreams[6-mer tokenizations] --> ObservedBits
  ObservedBits --> Decoder[Decoder / ECC / Score] --> Decision
end
Codeword --- Detector    <!-- key known to detector -->
Z_t --- Detector         <!-- no; only through DNA -->

```

Carbon vs FNS (Sec. 40,55). A key experiment: given the same context and key, compare watermarking standard Carbon (6-mer sampling) vs Carbon-FNS (base-level). Metrics: bits/base, sequence length needed, distribution cost (e.g. does FNS sampling affect k-mer distribution?), and edit robustness (FNS naturally immune to phase errors, as it emits raw bases). We predict FNS may yield at most 1 bit/base (since $2^1=2$ choices per base), vs Carbon-token could embed up to $H_b(q_t) \sim 1$ bit per token but less per base ($\leq 1/6$ bit per base in extreme?), actually no: token-level in theory can embed up to $\log_2(4096)=12$ bits per token, i.e. 2 bits/base maximum if partitions exploited fully. But practically entropy is low. We'll measure these. Whichever yields more *practical* watermark bits (information with error tolerance) will guide design.

Experiments (E0–E13). We define stable experiment IDs (E0...E13) and for each:

- **E0 (Synthetic Sampler):** No Carbon. Test samplers on known distributions (e.g. uniform, skewed, one-hot, etc). Verify invariants: distribution-preservation (TV distance near 0), correct symbol mapping, measured info bits, runtime.
- **E1 (Carbon Integration / Tokenizer Audit):** Load Carbon-3B (and 500M,8B); dump tokenizer vocab (manifest table), test encoding/decoding of all 6-mers, DNA state, "`<ooov>`", partial, etc. Log mass outside DNA vocab on many steps. Generate trace of logits for a few prompts to verify where to hook sampler. Deliverables: `tokenizer_audit.md`, `vocab_manifest.parquet`, `generation_trace.jsonl` (containing token probs), `model_manifest.json` (with versions: model_id, revision, transformers ver, torch, cuda, seed, hardware).
- **E2 (Carbon Channel Measurement):** For Carbon-3B (and 500M/8B subset) sample ~100k–500k DNA-generation steps. Log metadata (prompt type, model ver, top-k/p, temp), *probabilities mass* (dna_mass, bpe_mass, special_mass), *entropy_full*, *entropy_dna*, *effective_vocab*, *top-1 mass*, *top-2 mass*, *information_base*, etc. Also record random q_t for one or more random partitions to get $\mathbb{E}[q_t-0.5]$, predicted $\mathbb{E}[C_t; Y_t]$ for SAMP-A, and baseline bits. Summarize as histograms.
- **E3 (Distribution Preservation):** Generate matched sets: one with watermark sampler (keyed, C_t random), one baseline (unkeyed uniform C_t). For each config, compare N-gram distributions, GC-content, Carbon log-likelihood, independent GM likelihood, etc. Also train a classifier to see if can tell watermarked vs base. Key: verify no unintended bias.
- **E4 (Clean Detection):** Produce watermarked DNA (block lengths multiples of 6) of various lengths (e.g. 120bp up to 15kb). For each length, measure TPR at given FPR (start at 10^{-3} then lower). Show statistic separation for correct vs wrong key. Compute required length for e.g. 90% TPR. Evaluate worst-case (phase/orientation search).
- **E5 (Substitutions):** Apply random substitutions at rates 0.1%, 0.5%, 1%, 2%, 5%. Measure nucleotide error rate vs induced *token error rate* and *watermark bit error rate*. Run detection: TPR at fixed FPR vs substitution rate. Tabulate.
- **E6 (Insertions/Deletions):** Similarly, random insertion and deletion rates (0.1–5%). Compute fraction of tokens misaligned, phase recovery distance (how far from first error until re-sync if any), TPR drop. Compare naive global (SYNC-0) vs block/align strategies.

- **E7 (Cropping/Orientation):** Randomly crop 75%,50%,25%,10% of original (random offset) and test both strands and all 6 phases. Confirm detector picks correct phase and orientation. Compute TPR loss for shorter fragments.
- **E8 (Synchronization Comparison):** For a fixed sampler, compare SYNC-0 through SYNC-4 (and SYNC-5 if we implement). For each, plot TPR vs indel rate, false-positive overhead, and runtime. Show how many tokens need to be consumed to reliably detect under typical error.
- **E9 (Null/Wrong-Key Calibration):** Test detector on null DNA (random genomic or null generated) for multiple keys. Domains: natural DNA, unwatermarked Carbon outputs, permuted composition, repetitive sequences. Derive threshold for desired FPR. Use sufficient trials to bound FPR.
- **E10 (Unkeyed Distinguishability):** Train classifiers (logistic/regression, random forest, maybe a neural net) to distinguish watermarked vs baseline, without key. Features: k-mer counts ($k=1..6$), GC, repeats, Carbon logprob under model, etc. Ensure train/test separation by prompt and seed. Report AUC. Aim: ideally $AUC \approx 0.5$ (hard to tell).
- **E11 (Removal Attack):** Attacker tries generic edits to defeat watermark (still preserving plausible DNA). For each edit type and rate, find lowest edit budget to reduce detection below threshold. E.g. many random subs, then signal-loss. We refrain from complex genetic edits. This is a stress test rather than an optimized attack.
- **E12 (Sampler Baselines):** Repeat E2-E5 for multiple sampler algorithms (SAMP-A, SAMP-B, SAMP-C, DiPmark-inspired if feasible, any obvious baseline). Compare on a Pareto chart: bits/base vs detection length vs distribution distortion.
- **E13 (ECC/PRC Integration):** Characterize the final *watermark-symbol* channel: given observed bits (possibly with missing/unaligned segments), what effective error model (substitution, deletion, insertion of symbols) results? Then design an error-correcting code or PRC to embed a multi-bit message with blocklength L . This is optional (depending on capacity found). We will ensure not to overclaim; only after empirical channel rates are known.

For each experiment we define data schemas. For example, `carbon_channel.parquet` might have columns:

```
sequence_id, model_id, model_revision, mode (standard/fns), position_token,
position_bp, context_class, metadata_class,
temperature, top_k, top_p,
dna_mass, bpe_mass, special_mass,
entropy_full, entropy_dna, effective_vocab, max_prob, second_prob,
partition_id, q, abs_q_minus_half, predicted_err, information_token,
information_bp, selected_token_id, selected_token
```

The `detection_results.parquet` might include:

```
sequence_id, watermarked (bool), model_id, sampler, key_id,
length_bp, attack_type (none/sub/ins/del/crop), attack_rate,
orientation, phase, window_start, window_end,
score, p_value, decision (TP/FP)
```

And `edit_channel.parquet` with:

```
dna_sub_rate, dna_ins_rate, dna_del_rate,  
token_sub_rate, token_ins_rate, token_del_rate,  
phase_recovery_distance, watermark_symbol_error_rate
```

(These schemas guide data collection and analysis code.)

Project Roadmap (Secs. 79–82). We break into a *feasibility sprint (Week0)* and then a 12-week plan.

- **Week 0 (Day 1–7): Feasibility Sprint.**
- *Day1 (Tokenizer/Audit):* Load Carbon-3B, dump vocab, verify 4096 6-mers, test `<dna>`, `N`, partial lengths. Document tokenizer behavior. Output `tokenizer_audit.md` and `vocab_manifest.parquet`.
- *Day2 (Channel Measurement):* Sample ~100k steps from Carbon-3B (DNA mode). Compute entropy, mass ratios, random partition balances. Brief comparison to 500M/8B on a subset. Summarize in quick plots.
- *Day3 (Sampler Implementation):* Code basic SAMP-A, SAMP-B, SAMP-C (synthetic distributions only). Verify on toy $\$P(z)\$$. Ensure correctness (distribution match).
- *Day4 (Clean Detector Prototype):* Implement detector pipeline (no edits): phases, bits extraction, scoring (e.g. chi-squared for binary sequence vs random). Test on clean watermarked DNA short (e.g. 600bp). Measure detection length to hit fixed FPR (1e-3).
- *Day5 (Sub & Crop Tests):* Simulate substitutions (0.1–2%) on watermarked strings, measure symbol error and TPR drop. Generate a crop at random offset and test all phases/strands.
- *Day6 (Indel Test & Sync):* Simulate a few insertions/deletions, show that naive fails. Implement a sliding-window (SYNC-2) detector and see if we recover signal in at least one window. Possibly try a simple DP alignment for a short fragment (e.g. Viterbi on bit sequences).
- *Day7 (Compare and Decide):* Summarize results in table: for each method (A/B/C), list distribution-preservation error, clean detection length, sub impact, indel impact. Also compare standard vs FNS on a pilot. Decide: *CONTINUE*, *REQUIRES SYNC REDESIGN*, *USE FNS*, or *STOP*.
- **Weeks 1–2 (Integration & Baseline):** Finalize token handling, build generation loop. Collect large-scale Carbon-3B stats (E2). Ensure baseline distribution is correct (E3).
- **Weeks 3–4 (Channel & Capacity):** Complete E2 measurement (rate, entropy, bits). Compare 500M/8B. Decide which model is used for main. (Probably 3B for capacity; 8B maybe for scale test.)
- **Weeks 5–6 (Detector, Phase Search):** Refine detector (phase/orientation), clean detection tests (E4). Achieve stable TPR at desired FPR (e.g. 1e-6).
- **Weeks 7–8 (Edits, Sync):** Run substitutions (E5) and indels (E6). Implement sync strategies (E8). Evaluate their efficacy.
- **Week 9 (Security Calibration):** Null tests (E9), wrong-key (E9). Compute real FPR considering all hypotheses (Sec. 67–69). Train unkeyed classifier (E10).
- **Week 10 (Comparisons):** Baseline sampler comparison (E12), including possibly a baseline DiPmark or others. Analyze stealth metrics (GC, k-mer stats).
- **Week 11 (Sequence Quality):** Use Carbon's own evaluation suite (e.g. sequence recovery, GC bias) to ensure watermarking doesn't degrade output. Profile runtime overhead.

- **Week 12 (ECC/PRC, Final):** If needed, design an ECC layer (E13) for synchronization. Formally state and check proof obligations. Make final continue/go decision.

Compute Planning (Sec. 81). Many steps can use Carbon on GPU, but once detector is offline, can use CPU. For E1–E4, 3B model on GPU with fp16/bf16 is needed (batch across contexts). For channel measurement (E2) and watermark sampling, we need ~1M logits steps; this is significant (3B model at ~10k tokens/sec on good GPU, so ~100s of GPU hours unless parallelized). We budget accordingly. Detector and MC simulations can run on CPU. The one-week sprint focuses on CPU-quick tasks (vocab, small samples) and a short GPU run (~10k tokens).

Implementation Checklist (Sec. 49–50, 82–83). Key invariants to verify by unit tests and sanity checks:

- **Tokenizer:** round-trip on all 4096 kmers; unique IDs; consistency across models; `<dna>` tags required; `<oov>` on invalid; partial 6-mer padded with A; base vs DNA mode.
- **Distribution:** Samplers preserve $\sum P=1$; when disabled, our loop replicates vanilla transformer sampling exactly; no double-temperature application; no non-DNA leaks (if using DNA-only baseline). Test millions of draws on simple distributions.
- **Key:** Changing key yields independent stream; same key+seed yields reproducible.
- **Randomness:** Distinguish model randomness vs watermark randomness.

Failure Modes (Sec. 82): We will guard against issues like: missing `<dna>` tags, mis-handled partials (inserting silent A's), unintended BPE tokens in DNA mode, not padding properly, accidental inclusion of non-6mer tokens post-warp, forgetting eos. Also ensure we do sampling *after* temperature/top-k. For detector: off-by-one in phase shift, wrong reverse-complement mapping, bit extraction misalignment, ignoring multiple-testing.

Edge Cases (Sec. 83): We explicitly decide behavior for each, e.g.:

- Lowercase DNA: convert to uppercase (supported).
- N/IUPAC: map to `<oov>` (thus breaks watermark chain). Likely we should require input DNA has no N (or treat N as deletion?). Mention strategies.
- Whitespace/newlines: strip in normalization.
- Tags inside: disallow mixing `<dna>` inside DNA string.
- Metadata tokens in DNA: treat as invalid.
- Sequence <6bp: too short to contain a full token; likely abort.
- Crop < one block: also abort detection.
- Low-complexity repeats: may weaken statistical test, note this.

Figures (Sec. 84): We will include:

1. *System Architecture Diagram:* how sampler inserts into Carbon generation (mermaid flow).
2. *Vocab Distribution Chart:* pie chart of token mass (DNA vs BPE vs special) from E2 data.
3. *Effective Vocab/Entropy Distribution:* e.g. histogram of V_{eff} and entropy per position.
4. *Information Density:* bits/token and bits/base distribution under different settings.
5. *6-mer vs FNS Channel:* comparison plot of bits/base and TPR vs length for standard vs FNS (from experiments).
6. *Detection Scaling:* TPR vs fragment length at fixed FPR.
7. *Substitution/Indel Plots:* error rate vs damage, TPR drop.

8. *Sampler Comparison*: scatter of capacity vs distortion for SAMP-A/B/C.

9. *Sequence Quality*: comparison of model likelihood or GC drift.

(We may embed a few meaningful images from literature if they appear when browsing, but likely use mermaid for our pipelines.)

Decision Framework (Sec. 87): Based on results, we conclude:

- **CONTINUE**: If standard Carbon yields >0.1 bits/base (~ 0.6 bits/token) and detection $<$ few kbp, distributions match, etc.
- **CONTINUE BUT REDESIGN SYNCHRONIZATION**: If substitutions/crops ok but indels kill global parse. Then main challenge is coding.
- **USE GENERIC WATERMARK (ITS/EXP)**: If Kuditipudi schemes seamlessly integrate (they likely do). Possibly SAMP-A is enough.
- **USE FNS**: If 6-mer channel is too weak/impractical, but FNS gives much higher per-base info (up to 1 bit/base). Then baseline should switch to FNS.
- **STOP**: If Carbon's distribution is effectively deterministic (no entropy) or watermark is undone by trivial edits. This outcome is unlikely but considered.

We will carefully justify any threshold. For example, if we target $\text{FPR}=10^{-6}$ and $\text{TPR} \geq 90\%$ on 1k-bp with 1% errors, threshold selection should be based on null trials (Sec. 69). We explicitly state these choices: e.g.

$\alpha=10^{-6}$ per full search, requiring maybe $\text{TPR} \geq 0.8$ at 5% indels.

Output Files: The final report will include all equations (TeX math), code snippets (fenced), tables, mermaid diagrams. All citations will be formatted as `[ref†Lx-Ly]`. For example, Carbon description will cite [23], [47], etc. The report will be compiled to both Markdown and PDF, with a Table of Contents.

References: Key sources include the Carbon technical report on GitHub and HF site [21†L182-L189] [23†L230-L238], Carbon model cards [47†L191-L200] [57†L206-L214], and watermark literature [50†L28-L33] [51†L29-L37] [54†L23-L31]. All URLs and commit SHAs will be listed in an Appendix.

This plan aims to produce an actionable blueprint: another researcher should be able to follow each section to implement the system, conduct the experiments, and evaluate feasibility.